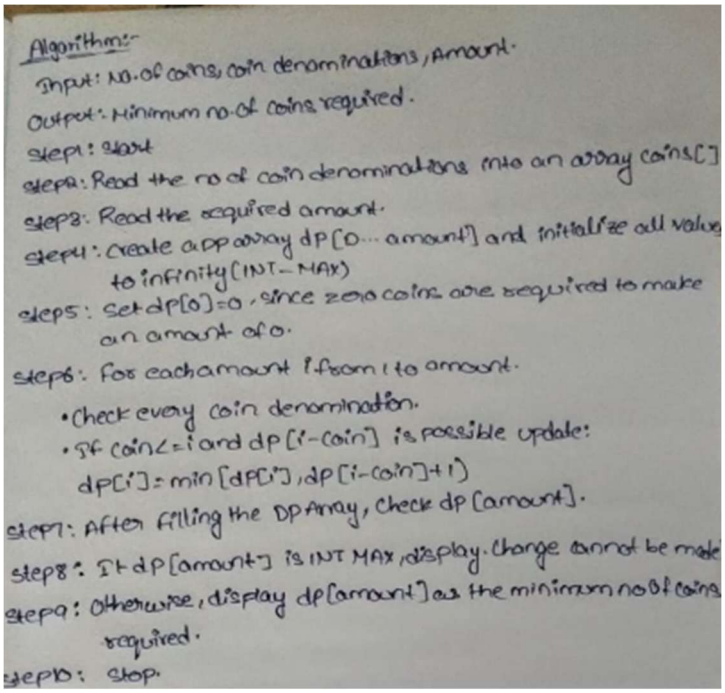
PRACTICAL -7

Aim: Implementation of making a change problem using dynamic programming.

Algorithm:



program:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n, amount;

    cout << "Enter number of coins: ";
    cin >> n;

    vector<int> coins(n);

    cout << "Enter the coin denominations:" << endl;
    for (int i = 0; i < n; i++) {
```

```
    cin >> coins[i];
        }

    cout << "Enter the amount: ";

    cin >> amount;

    // dp[i] = minimum number of coins needed to make amount i

    vector<int> dp(amount + 1, amount + 1);

    dp[0] = 0;

    // Dynamic Programming

    for (int i = 1; i <= amount; i++) {

        for (int j = 0; j < n; j++) {

            if (coins[j] <= i) {

                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }

    if (dp[amount] > amount) {

        cout << "Amount cannot be made using given coins." << endl;

    }

  else {
        cout << "Minimum number of coins required: "

             << dp[amount] << endl;
    }

    return 0;
}
```

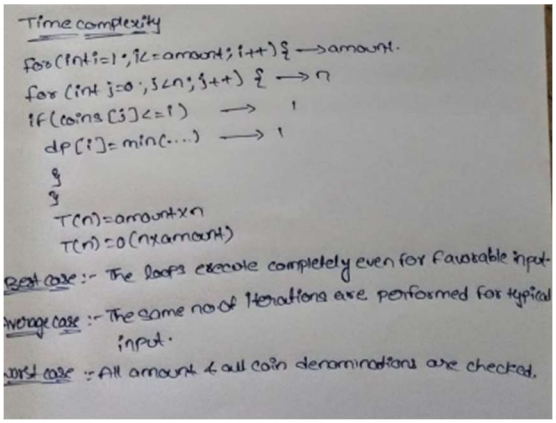
Name:B.Vennela
EnrollmentNo:92460118232

Output:

```
Enter number of coins: 7
Enter the coin denominations:
5
4
6
9
2
7
8
Enter the amount: 15
Minimum number of coins required: 2


=== Code Execution Successful ===
```

Time complexity:



Conclusion: : Thus, the Making Change problem was successfully implemented using the Dynamic Programming technique. The algorithm efficiently finds the minimum number of coins required to make a given amount by breaking the problem into smaller subproblems and storing their results. This approach avoids repeated calculations and improves the efficiency compared to a simple recursive solution.

Name:B.Vennela
EnrollmentNo:92460118232